CandySpeak Manual

Tony Rogvall

2026-07-27

Contents

Introduktion **2**
- Grundläggande koncept . . . 2

Språkbeskrivning **2**
- Deklarationer . . . 2
- Regler . . . 11
- States . . . 12
- Parts . . . 14
- Exekveringsmodell . . . 15
- Inbyggda funktioner . . . 16
- Operatorer . . . 16

Linux-verktyget **17**
- Installation . . . 17
- Användning . . . 17
- Simulerad tid (`-F`) . . . 19
- Generera inbäddad kod . . . 19

Avancerade koncept **19**
- Exekveringslägen . . . 19
- Api . . . 20
- Interaktivt läge . . . 20

Arduino-exempel **23**
- Blink - Blinkande LED . . . 23
- Knapp med LED . . . 24
- Debouncer - Avfångare . . . 24
- Toggle - Växla med knapp . . . 24
- Dimmer - PWM-styrning . . . 25
- Temperaturstyrning . . . 25
- Rinnande ljus - Knight Rider . . . 25
- Pulserande LED - Andning . . . 26
- Trafikljus . . . 26

Baka in ett program i firmware **27**

Skriva en sketch 28

PDF-generering **28**

Appendix: Snabbreferens **29**
Deklarationer 29
Buffertar 29
CAN 29
Modulinstansiering 29
Regler 30
Timer-funktioner 30
Kantdetektering 30
States 30
Parts 30
Interaktivt 31
Packa och packa upp 31

Introduktion

CandySpeak är ett reaktivt programmeringsspråk designat för inbyggda system och mikrokontrollers. Det kombinerar enkelhet med kraftfulla abstraktioner för att hantera sensorer, timers och I/O.

Språket är regelbaserat - varje rad beskriver VAD som ska hända och NÄR, inte HUR. Systemet evaluerar alla regler varje cykel och uppdaterar utgångar baserat på ingångar.

Grundläggande koncept

- **Variabler** - lagrar värden mellan cykler
- **Digitala I/O** - knappar, LEDs, reläer
- **Analoga I/O** - sensorer, PWM
- **Timers** - tidbaserade händelser
- **Moduler** - återanvändbara komponenter
- **States** - organisera regler till tillståndsmaskiner
- **Buffertar & ramar** - delad lagring, bitfält, pack/unpack
- **CAN** - en ram är en buffert med en adress; fälten är bitvyer in i den

Språkbeskrivning

Deklarationer

Deklarationer börjar med # och definierar programmets resurser.

Variabler

```
#variable <name>[:<bits>] [<type>] [= <value>]
```

Exempel:

```
#variable Counter:8 integer = 0
#variable Temperature float = 20.0
#variable Flag = 0
```

Bredden anges i **bitar, 1..32** (32 om den utelämnas); allt annat avvisas, och likaså en literal som inte får plats i typen — en felaktig bredd är avsevärt dyrare att hitta senare än vid deklarationen.

> **En bredd utan typ är TECKNAD.** `#variable c:4` är ett 4-bitars *tecknat* värde, alltså intervallet `-8..7`, och `c = 15` läses tillbaka som `-1`. Det följer regeln att en typlös variabel är `integer`, och gäller `#field` och `bind` på precis samma sätt. Vill man ha det raka intervallet `0..2^N-1` — flaggor, bitkvantiteter, råa signaler — får man säga det:
>
> ```
> #variable c:4 unsigned = 15      // 15, inte -1
> ```

Konstanter

```
#constant <name> = <value>
```

En konstant är ett namngivet läsbart värde som ligger fast från deklarationen. Till skillnad från en variabel kan den inte tilldelas av en regel; använd den för trösklar, skalfaktorer och gränser som ska läsas som namn i stället för magiska tal.

```
#constant Setpoint = 200
#constant Scale    = 4
```

Digitala I/O

```
#digital <name> [in|out|inout] [pullup|pulldown] [<port>:]<pin>
```

Port är valfritt (för mikrokontrollers med flera I/O-portar).

Exempel:

```
#digital Button in pullup 2
#digital Led out 13
#digital Relay out B:7
#digital Status inout C:4
```

Analoga I/O

```
#analog <name>[:<resolution>] [in|out] [pwm] [<port>:]<pin>
```

Upplösning är antal bitar (default 10).

Exempel:

```
#analog Sensor in A0
#analog Dimmer:8 out pwm 9
#analog HighRes:12 in A:1
```

Timers

```
#timer <name> <period_ms> [= 1]
```

`= 1` startar timern automatiskt vid programstart.

Exempel:

```
#timer Blink 500 = 1
#timer Debounce 50
#timer Timeout 5000
```

Buffertar

```
#buffer <name>:<size> [<type>] [in|out] [can <frame-id>]
```

En buffert är ett lagringsblock som variabler kan mappas in i. En vanlig variabel äger sitt eget värde; en buffert är *delad* — flera variabler kan bindas till olika bitfält i samma buffert, och bufferten kan också läsas och skrivas direkt byte för byte. Buffertar är byggstenen för ramar (CAN), bitpackning och överlappande data.

```
#buffer Frame:8            // 8 byte lagring (64 bitar)
#buffer Word:2             // 2 byte
```

> **Storleken anges alltid i byte.** En `#buffer` är en byte-behållare, oavsett transport — vanlig och CAN lika (en rams storlek *är* dess DLC, också byte). Vill du ha ett maskat sub-byte-värde? Det är en `#variable` (den bär sin egen bitbredd). Vill du ha en bit-vy in i en buffert? Det är ett `#field` eller ett `bind`-intervall.

En buffert kan deklareras inne i en modul, och då får varje instans sin egen lagring — se *Vad en modul får innehålla*.

En buffert kan användas direkt som en variabel. Hela bufferten är dess värde (upp till 4 byte — större buffertar nås byte för byte eller via bundna fält):

```
#buffer Status:1
Status = 200
```

Byte-åtkomst. Att indexera en buffert väljer hela byte:

```
Frame[0] = 52          // first byte
Frame[1] = 18          // second byte
X = Frame[0..1]        // bytes 0..1 as one value (little-endian)
```

Binda variabler (bitfält). Mappa en variabel på ett *bit*-intervall i en buffert med `bind`. Att skriva eller läsa en bunden variabel skriver/läser de underliggande bitarna — de aliaserar samma lagring. Bufferten måste vara deklarerad innan den binds.

```
#buffer Frame:3                          // 3 byte = 24 bitar lagring
#variable Speed:8     bind Frame[0..7]   // bits 0..7
#variable Rpm:10 big  bind Frame[8..17]  // bits 8..17, big-endian

Speed = 50             // writes into Frame's bits 0..7
```

> **Obs:** när man indexerar en *buffert* direkt är index en **byte** (`Frame[1]`), medan ett `bind`-intervall anges i **bitar** (`Frame[8..17]`). Direktindexering är för bekväm helbyte-åtkomst; `bind` är för att packa bitfält som är mindre än en byte.

Hur som helst måste intervallet hålla sig **inne i bufferten** och täcka högst **32 bitar** — ett bundet fält eller en `Buf[a..b]`-skiva som sträcker sig förbi slutet, eller som begär mer än 32 bitar, avvisas vid deklarationen.

CAN-ramar

En ram är en buffert med en adress. Att deklarera en ger en vanlig buffert en transport och ett ram-id; allt annat som gäller buffertar gäller sedan oförändrat — `bind`, `<<=`, `>>=`, byte-indexering.

```
#buffer F201:8 in  can 0x201      // 8 BYTES (the DLC), received
#buffer Fbig:64 out can 0x300     // CAN FD, transmitted
```

Riktningen är bussriktningen: `in`-ramar tas emot och packas upp, `out`-ramar sätts ihop och sänds. Id över `0x7FF` sänds automatiskt som extended (29-bitars) ramar.

Fält Signaler inne i en ram är bitvyer in i den. Det finns tre sätt att nå dem, och de är utbytbara — välj det som läser bäst:

```
#buffer F201:8 in can 0x201

// 1. #field -- a named field, declared against the frame
#field Speed:16 unsigned F201[0..15]
#field Temp:8   unsigned F201[16..23]

// 2. bind -- the same thing spelled as a variable
#variable Rpm:16 bind F201[24..39]

// 3. >>= -- no declaration at all, unpack on the fly
F201 >>= a:8 b:8 c:16 ? F201.rx
```

Ett `#field`-fält ärver riktning från ramen och behöver därför sällan en egen. Bitintervallet anges i **bitar**, räknat från bit 0 i byte 0, och värdet är little-endian om inte fältet säger `big`.

Eftersom alla fält i en ram är vyer in i en och samma buffert syns packningen direkt om man skriver ett fält och läser ett annat:

```
#buffer Out:8 out can 0x200
#field Lo:8  unsigned Out[0..7]
#field Hi:8  unsigned Out[8..15]
#field Both:16 unsigned Out[0..15]

Lo = 1
Hi = 2                  // Both now reads 0x0201
```

Att sända En `out`-ram sänds i slutet av varje cykel där något av dess fält ändrats. Det är en **händelse-PDO** och kräver ingen extra syntax — att tilldela ett fält är triggern:

```
Speed = v               // frame 0x200 goes out this cycle
```

För en **cyklisk PDO** — sänd enligt schema oavsett om något ändrats — finns det inget som gör ramen smutsig, så be om sändningen uttryckligen med `.tx`:

```
#timer Beat 100
Beat = 1
Beat = 1      ? timeout(Beat)
Out.tx = 1    ? timeout(Beat)     // send every 100 ms regardless
```

`.dlc` styr hur många byte som går ut. Den börjar på ramens deklarerade storlek och klamras till den:

```
Out.dlc = 3             // send 3 bytes instead of 8
```

Att ta emot En mottagen ram levereras in i bufferten och dess fält uppdateras. `.rx` är sann i **exakt en cykel** — den där de mottagna byten blivit läsbara:

```
println("speed=", Speed) ? F201.rx
```

På mottagarsidan läser `.dlc` tillbaka hur många byte avsändaren faktiskt sände, och `.id` ger ram-id:t. Båda fungerar genom ramen eller genom vilket fält som helst av den (`Speed.id` och `F201.id` är samma sak).

För att reagera bara när en *signal* ändras i stället för på varje ram, guarda på `changed()` — en ram som upprepar sitt innehåll ger då ingen utskrift:

```
println("speed changed") ? changed(Speed)
```

> **Encykelsregeln.** `.rx`, `changed()` och `<-` fyrar alla i den cykel där det nya värdet fortfarande ligger i skuggkopian, och regler läser den *committade* sidan. Att guarda en utskrift direkt på dem visar därför **föregående** värde. Låt triggern gå via en variabel så hamnar de i fas — se *Exekveringsmodell*:
>
> ```
> #variable fresh = 0
> fresh = F201.rx
> println("speed=", Speed) ? fresh      // now in phase
> ```
>
> En `bind`ad variabel har inte det problemet: den *aliaserar* rambitarna i stället för att kopiera dem, så den är redan korrekt i `.rx`-cykeln. En uppackning (`>>=`) skriver kopior, som landar nästa cykel som all annan skrivning.

Ram-parts

Part	Riktning	Betydelse
`.id`	läs	CAN-ramens id
`.rx`	läs	en ram har kommit och är läsbar denna cykel (bara en cykel)
`.tx`	läs/skriv	skriv 1 för att tvinga fram en sändning

Part	Riktning	Betydelse
`.dlc`	läs/skriv	byte att sända / byte senast mottagna
`.dir`	läs/skriv	bussriktning (`in` = 1, `out` = 2)

`.dir` är en egenskap hos bufferten och fungerar därför även på en vanlig `#buffer`.

Observera att ram-parts ligger på bufferten i stället för i en value-slot, så till skillnad från `.period` är de **inte** skuggade: `F.dlc = 3` följt av en läsning av `F.dlc` i samma cykel ger 3.

Att koppla till en buss På Linux går ramarna till SocketCAN, valt med `--can`:

```
sudo ip link add dev vcan0 type vcan && sudo ip link set up vcan0
./csp --can=vcan0 -c 0 examples/can_input.csp
# from another shell:
cansend vcan0 201#2A3412785634120000
candump vcan0
```

Utan `--can` parsar deklarationerna ändå och programmet kör ändå; ramarna går bara ingenstans. På Arduino aktiveras backenden per bräda genom att definiera `CSP_HAS_CAN` i dess Makefile (den förutsätter arduino-CAN-API:t och en transceiver).

Se `examples/can_input.csp`, `examples/can_output.csp` och `examples/can_pack.csp` för färdiga program.

> **Begränsningar idag.** Ett fält får börja på vilken bit som helst i en 64-bytes FD-ram (`0..511`) och vara **1..32 bitar** brett — värdet det ger är en 32-bitars behållare, så en bredare signal måste delas i två fält. En deklaration utanför de gränserna, eller en som sträcker sig förbi ramens slut, avvisas istället för att wrappa. RTR-ramar stöds inte.

Packa och packa upp ramar

`bind` ger ett *namngivet, permanent* fält. När man bara vill sätta ihop eller plocka isär en ram i farten — utan att deklarera en variabel per fält — använder man pack (`<<=`) och unpack (`>>=`).

Pack — `<buffert> <<= <fält> <fält> ...` skriver flera bitfält till en buffert i en regel. Fälten är **blanksteg-separerade** och fyller **stigande** bit-offset; vart och ett maskas till sin bredd. Ett fält är `<uttryck>[:<bitar>]`; bredden defaultar till variabelns deklarerade bredd när den utelämnas.

```
#buffer Frame:8
#variable A:3 = 5
#variable B:2 = 3
#variable C:3 = 6

Frame <<= A B C            // 5 | (3<<3) | (6<<5) = 221
```

Fält kan vara uttryck och literaler med uttrycklig bredd:

```
Frame <<= (X+4):3  X:2  6:3   // 5 | (1<<3) | (6<<5) = 205
```

Unpack — `<buffert> >>= <var> <var> ...` är den exakta spegeln: varje variabel tar emot nästa bitfält ur bufferten, lägsta offset först.

```
#variable RA = 0
#variable RB = 0
#variable RC = 0

Frame >>= RA:3 RB:2 RC:3   // RA=5, RB=3, RC=6
```

Pack följt av unpack är en ren tur och retur: `<<=` kodar, `>>=` avkodar, och bredderna matchar fält för fält. Jämfört med `bind` håller pack/unpack inget tillstånd — de är en engångskodning man placerar i en regel, idealisk för tillfällig ramhantering strax före sändning eller direkt efter mottagning.

Moduler

Moduler grupperar relaterad funktionalitet för återanvändning. En modul är en mall som kan instansieras flera gånger med olika konfigurationer.

```
#module <name>
  <declarations>
  <rules>
#end
```

Modulinstansiering

```
#<ModuleName> <instance> [<init>]*
```

Där `<init>` kan vara:

Form	Betydelse
`fält = värde`	Sätt fältets **värde** (statiskt, körs en gång)
`fält.part = värde`	Sätt ett **attribut** en gång — `D.pin`, `D.port`, `T.period`
`fält <- uttryck`	Reaktiv koppling (uppdateras när uttrycket ändras)

De två första är olika saker: `D = 2` skriver ett värde in i `D`, medan `D.pin = 2` placerar pinnen. Att konfigurera hårdvara är alltid `.part`-formen.

Init-uttryck kan blandas fritt. Fält markerade `in` måste initieras.

Semantik för objektinitiering De två formerna beter sig olika, med flit:

- **Statisk (`=`, `.part =`) körs *en gång*.** Dessa initierare exekverar i instansens implicita **INIT**-tillstånd och sedan går objektet över till **NORMAL** (se *States*). Att skriva dem varje cykel vore slöseri och skulle hålla konfigurationsutgångar permanent “smutsiga”, så de är engångs.

- **Reaktiv (<-) är en stående koppling.** Den omvärderas när en ingång i högerledet ändras. Den **seedas också en gång vid uppstart**: första cykeln fyrar varje `<-` en gång så fältet får ett startvärde — även när högerledet är en konstant eller en global som aldrig ändras sedan. Så `x <- G + 1` ger `x` värdet `G + 1` omedelbart och följer sedan senare ändringar av `G`.

Fält kontra globaler. En `<-`-initierare får läsa en **global** fritt:

```
#M m1 X <- G + 1          // G is a global -> fine
```

För att uttrycka ett samband *mellan fält i ett objekt* (eller mellan två instanser), skriv en regel i stället för en initierare — antingen inne i modulen, eller globalt med punkt-notation på instanserna:

```
Total = m1.Out + m2.Out    // relate fields across instances with a rule
```

Vad en modul får innehålla En modul är en mall. Inne i `#module ... #end` kan du deklarera samma resurser som på toppnivå — `#variable`, `#digital`, `#analog`, `#timer`, `#buffer`, `#field`, `#constant` — plus modul-lokala `#states`, och de regler (inklusive `#in <state>`-block) som verkar på dem. Varje instans får sin egen kopia av varje deklarerad medlem och sitt eget tillstånd, inklusive sin egen buffertlagring:

```
#module Frame
  #buffer B:16                       // one buffer PER INSTANCE
  #variable lo:8 bind B[0..7]
  #variable hi:8 bind B[8..15]
#end

#Frame f1 lo=1 hi=2                  // f1.B is 0x0201
#Frame f2 lo=9 hi=8                  // f2.B is 0x0809, independent
```

Moduler kan läsa globaler. En modulkropp ser allt som deklarerats ovanför den — konstanter, variabler, timers, buffertar. En medlem med samma namn **skuggar** den globala, så att lägga till en global i efterhand kan inte knäcka en modul som råkar använda det namnet.

```
#constant GREEN = 0x07E0
#buffer Live:16                      // shared by every instance

#module Pixel
  #analog P out 9:0
  P = GREEN ? ...                    // global constant
  P = Live  ? ...                   // global buffer, one for all instances
#end
```

Att binda en medlem till en *global* buffert delar den alltså mellan instanser, medan bindning till en *medlems*buffert ger varje instans sin egen — vilket av de två man får följer av var bufferten är deklarerad.

Exempel: Full Adder

```
#module Add
#variable A:1 in
#variable B:1 in
#variable Cin:1 in
#variable S:1 out
#variable Cout:1 out

S = A ^ B ^ Cin
Cout = (A & B) | (Cin & (A ^ B))
#end

#Add a0 A=1 B=1 Cin=0
#Add a1 A=0 B=0 Cin <- a0.Cout
#Add a2 A=0 B=1 Cin <- a1.Cout
```

Här har `a0` statiska ingångar, medan `a1` och `a2` kopplar sin carry-ingång reaktivt från föregående adders carry-utgång.

Pin-tilldelning för I/O När en modul innehåller `#digital` eller `#analog` kan pinnen lämnas som en platshållare i modulen och tilldelas per instans med `.pin`:

```
#module Button
#digital Pin in pullup 0     // placeholder pin
#variable Out = 0
#timer T 50

T = 1 ? Pin != Out
Out = Pin ? timeout(T)
#end

#Button btn1 Pin.pin=2       // assign the PIN at instantiation
#Button btn2 Pin.pin=3
#Button btn3 Pin.pin=4
```

> **`Pin = 2` är inte samma sak.** Ett bart `fält = värde` sätter fältets **värde**, precis som för en variabel — för en 1-bitars digital skriver `Pin = 2` värdet 1 och lämnar pinnen på 0. Använd `Pin.pin = 2` för att placera den, och `Pin.port = 1` för porten. Samma sak gäller `#analog`.

Detta gör moduler återanvändbara över olika hårdvarukonfigurationer. Det överrider också en default som givits i modulen:

```
#module Led
#digital Out out 13          // default pin 13
...
#end

#Led led1                    // uses default pin 13
#Led led2 Out.pin=12         // override to pin 12
```

Åtkomst till modulfält Använd punktnotation för att komma åt en modulinstans fält:

```
MainLed = btn1.Out       // read debounced output
```

Regler

Regler har formen:

```
<actions> [? <condition>]
```

Åtgärderna utförs när villkoret är sant. Delen `? <villkor>` är valfri — en regel utan villkor körs varje cykel (den är alltid sann).

Tilldelning

Vanlig tilldelning - utvärderas varje cykel:

```
Led = 1 ? Button == 0
Counter = Counter + 1 ? Timer
```

Reaktiv tilldelning

Med `<-` körs regeln endast när någon variabel i högerledet ändras:

```
Output <- Input * 2
```

Regeln ovan körs bara när `Input` ändras, inte varje cykel.

Uppstartsseed. En `<-`-regel evalueras också **en gång på första cykeln**, så målet alltid får ett startvärde — även om högerledet är en konstant eller en ingång som aldrig ändras sedan. Det gör att `Output <- Input * 2` är korrekt från allra första cykeln, inte först efter nästa ändring av `Input`.

Med villkor - regeln körs när RHS-variabel ändras OCH villkoret är sant:

```
Filtered <- RawValue ? RawValue > Threshold
```

Flera åtgärder

Separera med komma:

```
Led = 1, Counter = Counter + 1 ? Button == 0
```

Villkor

Villkor kan vara:

- Jämförelser: `==`, `!=`, `<`, `>`, `<=`, `>=`
- Logiska: `&&` (och), `||` (eller), `!` (icke)
- Timer-funktioner: `timeout(T)`, `elapsed(T)`, `progress(T)`
- Specialfunktioner: `changed(x)`, `rising(x)`, `falling(x)`

Exempel

```
Led = ~Led ? timeout(Blink)
```

Växla LED vid varje timeout.

```
Output = 1, T = 1 ? Input && !Output
```

Sätt Output och starta timer T när Input blir hög.

States

States gör en platt lista av regler till en tillståndsmaskin. Du räknar upp tillstånden och grupperar sedan reglerna under det tillstånd de hör till. Det håller stora program läsbara och låter runtime hoppa över hela block av regler som inte är aktiva.

```
#states A B C
```

Tre tillstånd finns alltid implicit: **INIT** (som man går in i vid uppstart, för engångsinitiering), **NORMAL** (default körläge) och **FAILSAFE** (se nedan). Dina egna tillstånd läggs till ovanpå; du kan räkna upp fler när som helst med en till `#states`-rad.

Regler knyts till ett tillstånd med ett `#in <state> ... #end`-block:

```
#in A
    X = 1 ? Y < Z              // stay in A while this holds
    State = B ? X > Z          // transition to B
#end

#in B
    Z = Z + 1, State = A   // do work, then go back to A
#end
```

En övergång är bara en tilldelning till det reserverade fältet `State`. Att gå in i INIT, göra grunduppsättning och sedan gå vidare är den vanliga formen:

```
#in INIT
    Count = 0, State = NORMAL
#end
```

Mental modell. För att resonera om beteendet kan man läsa `#in S` som att `&& State == S` läggs till varje regel i blocket. De två formerna nedan beter sig likadant:

```
#in A
    Y = 1, State = B ? X > Z
#end
```

```
Y = 1, State = B ? X > Z && State == A
```

Men det är en mental modell, inte en källkodsomskrivning. Blocket kompilerar till en **grind** framför gruppen: ett tillståndstest som hoppar över hela blocket när tillståndet inte matchar, så ett inaktivt tillstånd kostar en enda kontroll hur många regler det än håller — det är själva poängen med att gruppera dem. Ett test per regel ligger kvar vid sidan av för den reaktiva vägen, som når regler individuellt i stället för att gå igenom blocket.

Du kan lägga till hur många regler som helst i ett tillstånd, och dela upp ett tillstånd över flera `#in S`-block — de ackumuleras.

Flera tillstånd samtidigt. Ange fler än ett tillstånd för att köra ett block i *något* av dem — tillstånden OR:as ihop. Delad infrastruktur (en heartbeat, en panikknapp, en timer-omstart) som måste köra över flera faser läggs här istället för att kopieras in i varje block:

```
#in red redyellow green yellow
    Phase = 1 ? timeout(Phase)     // starta om timern i varje kör-fas
    State = FAULT ? Panic          // panikknappen, kollad i alla
#end
```

Läs `#in A B C` som `&& (State == A || State == B || State == C)` på varje regel i blocket.

Regler utan `#in` — NORMAL+. En top-level-regel som *inte* ligger i något `#in`-block körs som standard i de två inbyggda drift-tillstånden **INIT** och **NORMAL** — aldrig i ett speciellt tillstånd. Så ett program helt utan tillstånd bara kör (det sitter i INIT/NORMAL), och i samma stund en tillståndsmaskin stegar in i ett *speciellt* tillstånd — ett användartillstånd, eller reserverade **FAILSAFE** — så *tystnar* de lösa globala reglerna istället för att läcka in i det. Det är vad som håller FAILSAFE till en ö: bara `#in FAILSAFE` (och block som uttryckligen listar det) kör där. Vill du köra en global regel i ett speciellt tillstånd också, namnge det tillståndet med `#in`.

FAILSAFE är klibbigt. Det är det utpekade säkra tillståndet, och när `State` väl håller det kan **ingen regel lämna det** — bara en reset gör det. En regel som tilldelar något annat ignoreras helt enkelt, så en skakig vaktregel kan inte studsa ut enheten ur en säker konfiguration. Tillsammans med tystnadsregeln ovan är det vad som håller FAILSAFE till en ö: lösa globala regler slutar köra där, och bara `#in FAILSAFE` (plus block som namnger det) har något att säga till om över utgångarna.

> Så ser FAILSAFE ut idag, och det är med flit tunt. Den tänkta formen är en `#module FAILSAFE`, kompilerad som en **egen ROM-image** med egna deklarationer, eget `#in INIT` och egen EEPROM-bank — så att en enhet kan bära flera banker, varav en är den säkra, och växlingen blir en rebase istället för en tillståndsövergång. Skriver du din säkerhetslogik som ett självförsörjande block redan nu flyttar den över rent.

States i moduler. En modul kan deklarera sina egna lokala tillstånd. Varje instans bär sitt eget `State`, så instanserna stegar genom maskinen oberoende av varandra:

```
#module Blinker
  #digital Led out
  #timer   T 500
  #states  on off

  #in INIT
    T = 1, State = off
  #end
  #in off
    Led = 1, State = on ? timeout(T)
```

```
  #end
  #in on
    Led = 0, State = off ? timeout(T)
  #end
#end

#Blinker b1 Led.pin=12
#Blinker b2 Led.pin=13
```

Parts

Varje resurs bär inte bara ett *värde* utan också en uppsättning **attribut** — pinnen hos en digital, perioden hos en timer, riktningen hos en port. Dessa attribut nås med punktnotation som **parts**, och de flesta kan både läsas och skrivas av en regel.

`<resource>.<part>`

Part	Gäller	Betydelse
`.val`	alla	det vanliga värdet (default när ingen part anges)
`.pin`	digital / analog	pin-nummer
`.port`	digital / analog	portnummer
`.dir`	digital / analog / buffert	riktning (in/out)
`.pwm`	analog	PWM-flagga
`.endian`	bundet fält / `#field`	byteordning (`0` native, `1` little, `2` big)
`.pullup`	digital	pull-up på
`.pulldown`	digital	pull-down på
`.period`	timer	timerperiod i ms
`.fired`	timer	timeout inträffade denna cykel
`.id`	CAN-ram / fält	ram-id:t
`.rx`	CAN-ram / fält	en ram har kommit och är läsbar denna cykel
`.tx`	CAN-ram / fält	skriv 1 för att tvinga fram en sändning
`.dlc`	CAN-ram / fält	byte att sända / byte senast mottagna

Ett CAN-fält svarar för sin ram, så `Speed.id` och `F201.id` är samma sak. Ram-parts ligger på bufferten i stället för i en value-slot, så till skillnad från de övriga är de **inte** skuggade — att skriva `.dlc` och läsa tillbaka den i samma cykel ger det nya värdet.

Exempel — läsa och skriva attribut som vilket värde som helst:

```
D.pin    = 17             // reassign a digital's pin at runtime
T.period = 500            // change a timer's period
Per      = T.period       // read it back
Ready    = 1 ? T.fired    // use a timer part in a condition
```

Att skriva `.pin`, `.port`, `.dir`, `.pullup` eller `.pulldown` konfigurerar om hårdvaran — pinnen ställs i sitt nya läge vid cykelns slut, så nästa läsning eller skrivning använder det.

`.pin` lämnar inte tillbaka pinnen den flyttar ifrån. En digital som drev pinne 4 hög och flyttas till pinne 7 konfigurerar pinne 7 och lämnar pinne 4 som utgång, fortfarande hög — och inget namn i programmet pekar på den längre, så ingenting kan rätta till det. Sätt den gamla pinnen till en säker nivå före flytten, eller låt bli att flytta. Detsamma gäller `.port` på ett kort som adresserar pinnar per port.

Att sätta en part i objektinit är det idiomatiska sättet att konfigurera en instans:

```
#Blinker b1 Led.pin = 12   // configure the instance's pin once, in INIT
```

Exekveringsmodell

En cykel fungerar så här.

- **Cykel.** Runtime läser ingångar, evaluerar regler och skriver utgångar om och om igen. Ett varv är en *cykel*; `cycle()` ger dess nummer.
- **Atomär commit.** Inom en cykel ser läsningar de värden som committades i slutet av *föregående* cykel, och skrivningar samlas i en skuggkopia. I slutet av cykeln committas alla ändringar på en gång. Ordningen som reglerna är skrivna i ändrar alltså inte resultatet, och en regel ser aldrig en annan regels halvfärdiga uppdatering. När två regler skriver samma fält i en cykel vinner den sista skrivningen vid commit.
- **Ändringsmängd.** Commit registrerar vilka fält som faktiskt ändrades. Det är vad som driver `changed(x)` och `<-`-triggern, och vad det reaktiva läget använder för att avgöra vilka regler som ska köras om.
- **Sekventiellt kontra reaktivt.** Sekventiellt läge evaluerar varje regel varje cykel. Reaktivt läge evaluerar bara regler vars trigger fyrade. Beroendegrafen byggs av det som står bakom `?`, och av båda sidorna i `X <- Uttryck ? Villkor`. En regel med varken villkor eller `<-` — ett bart `Y = X` — har alltså **ingen trigger** och körs bara i den första (seedande) cykeln. Det är med flit, inte en begränsning: reaktivt läge kör det du sagt åt det att bevaka.

Encykelsregeln. Läsningar ser föregående commit; skrivningar landar vid nästa. Allt som *triggar på ändring* — `changed(x)`, `<-`, en rams `.rx` — fyrar i den cykel där det nya värdet fortfarande ligger i skuggkopian. I den cykeln läser alltså värdet självt fortfarande som det **gamla**:

```
X = 1                       // X changes 0 -> 1
println(X) ? changed(X)     // prints 0, not 1
```

Det är transaktionsmodellen tillämpad konsekvent, inte ett specialfall: en ingång som samplas denna cykel blir läsbar nästa, och det gäller en ändrad variabel också. Två följder värda att känna till:

- För att agera på en ändring *med det nya värdet*, låt triggern gå via en variabel. Den fördröjs exakt lika mycket som värdet, vilket lägger dem i fas:

  ```
  #variable fresh = 0
  ```

```
fresh = changed(X)
println(X) ? fresh         // prints 1
```

- En källa som ändras **exakt en gång** triggar aldrig om, så `Y <- X` fångar värdet från före ändringen och behåller det. Med en källa som ändras löpande visar sig samma mekanism bara som en cykels eftersläpning, vilket är ofarligt.

Värden som *aliaserar* i stället för att kopiera är undantagna: en `bind`ad variabel är samma lagring som sin buffert och är därför korrekt omedelbart.

Inbyggda funktioner

Funktion	Beskrivning
`timeout(T)`	Sant en cykel när timer T går ut
`elapsed(T)`	Millisekunder sedan T startade
`progress(T)`	0-100, hur långt timern kommit (%)
`changed(x)`	Sant om x ändrades denna cykel
`rising(x)`	Sant vid 0→1 övergång på digital ingång
`falling(x)`	Sant vid 1→0 övergång på digital ingång
`cycle()`	Nuvarande cykelnummer
`tick()`	Nuvarande tid i millisekunder
`abs(x)`	Absolutvärde (heltal eller flyttal, följer argumentet)
`sign(x)`	Tecknet på x: -1, 0 eller 1
`min(a,b)`	Minsta värdet (heltal eller flyttal, följer argumenten)
`max(a,b)`	Största värdet (heltal eller flyttal, följer argumenten)
`clip(x,lo,hi)`	Klam x till intervallet lo..hi (heltal eller flyttal)
`trunc(x)`	Flyttal till heltal, mot noll
`round(x)`	Flyttal till heltal, närmaste (halva bort från noll)
`print(x)`	Skriv ut värde (upp till 4 argument)
`println(x)`	Skriv ut med radbrytning (0 till 4 argument)
`latch(b)`	Håll utgångar (1) eller släpp igenom (0)

Operatorer

Prioritet	Operator	Beskrivning
Högst	`~ ! -`	Bitvis NOT, logisk NOT, negation
	`* / %`	Multiplikation, division, modulo
	`+ -`	Addition, subtraktion
	`<< >>`	Bitskift
	`< <= > >=`	Jämförelse
	`== !=`	Likhet
	`&`	Bitvis AND

Prioritet	Operator	Beskrivning
	^	Bitvis XOR
	|	Bitvis OR
	&&	Logisk AND
Lägst	||	Logisk OR

Linux-verktyget

Installation

```
git clone <repo>
cd candyspeak
make
```

Användning

```
./csp [options] [file.csp]
```

Flaggor

Flagga	Beskrivning
`-h`	Visa hjälp
`-i`	Interaktivt läge
`-n`	Endast parsning, kör ej
`-C`	Visa kompilerad C-kod
`-c N`	Max antal cykler
`-T MS`	Max körtid i millisekunder
`-r`	Reaktivt läge (av om den inte anges)
`-Q`	Spåra variabelvärden
`-P`	Debug parser
`-R`	Debug resultat
`-S`	Debug scanner/tokenizer
`-d`	Debug (allmänt)
`-L erlang`	Spårutdata i Erlang-format
`-s <fil>`	Skriv en tillståndsspårning per cykel (Erlang-format) till fil
`-p <fil>`	Skriv parser-debug till fil
`-O <fil>`	Skriv en objektdump till fil
`-e <fil>`	EEPROM-fil (persistent tillstånd) att använda (default `eeprom.db`)
`--no-eeprom`	Lägg inte på de sparade EEPROM-patcharna vid boot
`-I <fil>`	Mata ingångar från fil (realtid, cykelstämplade rader)
`-F <fil>`	Mata ingångar med **simulerad tid** (se nedan)

Flagga	Beskrivning
`-b`	Starta pausad; inspektera, sedan `/resume` (implicerar `-i`)
`--can=IFACE`	SocketCAN-interface för `#field`-ramar (t.ex. `vcan0`)
`-m N[k]`	Användbar kodminnesbudget i byte
`-M N[k]`	Totalt RAM som den simulerade brädan har
`-U N[k]`	RAM som systemet och länkade bibliotek tar
`-E N[k]`	Simulerad EEPROM-kapacitet (0 = obegränsad)
`--board=NAMN`	Simulera en uppmätt bräda: `mega`, `mkrzero`

Simulera en bräda

Värdverktyget kan låtsas ha en mikrokontrollers minne, så att `/memory` visar siffror som betyder något innan man flashat något:

```
./csp --board=mega -i program.csp     # measured RAM/EEPROM for an ATmega2560
./csp -M 8k -U 2700 -i program.csp    # or set the numbers by hand
```

`--board` fyller i `-M`, `-U` och `-E` från siffror uppmätta på riktiga firmware-byggen (generera om dem med `make boards`). `/memory` visar sedan hur mycket av den brädans RAM programmet faktiskt skulle ta.

EEPROM läses vid uppstart

Startad **utan programfil** beter sig `./csp` som ett kort som kommer ur reset: det laddar det inbakade ROM-programmet och lägger sedan på det som `/save` skrev till EEPROM-filen (`eeprom.db` om inte `-e` säger annat), så de deklarationer, regler och disables du sparade förra gången är tillbaka. Ger du det en programfil hoppas överlagringen över — att namnge en fil betyder "kör *den här*". `--no-eeprom` hoppar över den också, för en medvetet ren boot eller ett upprepbart test.

Exempel

Kör ett program:

```
./csp program.csp
```

Visa kompilerad kod:

```
./csp -C -n program.csp
```

Kör max 100 cykler med spårning:

```
./csp -c 100 -Q program.csp
```

Interaktivt läge:

```
./csp -i
```

Simulerad tid (`-F`)

Med `-F` körs programmet mot en **virtuell klocka** i stället för väggklockan, vilket gör timer- och ingångstajming helt deterministisk och ögonblicklig (ingen verklig väntan). Varje ingångsrad stämplas med en absolut virtuell tid i millisekunder:

```
<time_ms>  <var>=<value>  [<var>=<value> ...]
```

En rad tillämpas när den virtuella klockan når dess tid. Mellan raderna hoppar klockan direkt till nästa händelse (nästa ingångsrad eller nästa timeout), så en 1-sekunderstimer fyrar efter ett steg i stället för tusen cykler — samtidigt som den alltid går fram minst ett tick per cykel så `cycle()` och tidsberoende logik alltid ser tiden röra sig framåt.

Exempel — mata en sensor vid två virtuella tider och låt en timer gå ut:

```
# stimulus.dat
30   Sensor=10
70   Sensor=21
```

```
./csp -c 100 -s trace.txt -F stimulus.dat program.csp
```

`csp_time_ms()` returnerar den virtuella klockan i det här läget, så `timeout(T)`, `elapsed(T)` och `progress(T)` beter sig alla deterministiskt. Det är det rekommenderade sättet att skriva repeterbara tester för timers och analog/digital ingång.

Generera inbäddad kod

För att generera C-kod för Arduino eller annan mikrokontroller:

```
./csp -C -n program.csp > program_code.h
```

Inkludera sedan denna header i ditt Arduino-projekt tillsammans med `csp.h` och runtime-filerna.

Avancerade koncept

Exekveringslägen

Som default evalueras varje regel varje cykel. Reaktivt läge är en valfri optimering för stora program där få saker ändras per cykel.

Oavsett läge tillämpas ändringar inom en cykel atomärt vid dess slut (se *Exekveringsmodell*): regler ser aldrig varandras halvfärdiga uppdateringar och evalueringsordningen spelar ingen roll.

Reaktivt läge (`-r`)

Default: AV — ange `-r` för att slå på det.

I reaktivt läge evalueras bara regler vars ingångar har ändrats. Detta är effektivare när:

- Få variabler ändras varje cykel
- Programmet har många regler
- Systemet har begränsad CPU

Det kostar en del minne för beroendegrafen.

Vad som får en trigger. Grafen byggs av det som står bakom `?`, och av båda sidorna i `X <- Uttryck ? Villkor`. En regel som varken har villkor eller `<-` bevakar ingenting, så i reaktivt läge körs den bara i den första (seedande) cykeln:

```
Y = X            // sequential: every cycle. reactive: first cycle only.
Y <- X           // reactive: whenever X changes
Y = X ? cond     // reactive: whenever cond's inputs change
```

Skriv reglerna på det reaktiva sättet så når båda lägena samma committade tillstånd. Regelordningen respekteras i båda: när två regler skriver samma fält vinner den som står sist — vilket är det som gör det förutsägbart att patcha ett körande system.

```
./csp -r program.csp     # reaktivt läge
./csp program.csp        # evaluera alla regler (default)
```

Api

Stödkonfiguration och defaultvärden för reaktivt läge finns i `csp_config.h`:

```
#include "csp_config.h"
```

Konfigurationsparametrarna listas nedan. Var och en måste definieras till antingen 1 eller 0.

```
REACTIVE_DEFAULT
SUPPORT_REACTIVE

USE_STATISTICS
USE_FIXPOINT
```

Programmerarens API för att ändra dessa vid körning — till exempel i en Arduino-sketch under `setup()`:

```
extern int     csp_set_reactive(csp_rt_t*, int onoff);
extern int     csp_set_latch(csp_rt_t*, int onoff);
```

Interaktivt läge

Starta interaktivt läge med `-i`:

```
./csp -i
./csp -i program.csp    # load program first
```

Interaktiva kommandon

Kommando	Beskrivning
`/help`	Visa kommandon
`/list`	Lista deklarationer och regler, märkta `[ROM]`/`[RAM]`
`/state`	Visa nuvarande värden, grupperade per objekt
`/memory`	Visa RAM-användning per kategori (hur mycket som är kvar)
`/reset`	Återställ till initialvärden
`/clear`	Släng RAM-patchar, behåll ROM-programmet
`/pause`	Frys exekveringen; prompten lever kvar
`/resume`	Fortsätt (bygger om först om programmet ändrats)
`/live`	Frys *reglerna* men låt I/O fortsätta
`/latch on`	Håll utgångar (frys nuvarande värden)
`/latch off`	Släpp igenom utgångar (normal drift)
`/commit`	Verkställ väntande värden
`/save`	Spara tillstånd till EEPROM-fil
`/load`	Ladda tillstånd från EEPROM-fil
`/quit` (eller `/exit`)	Avsluta

`/state` inleds med en statusrad som visar var man befinner sig:

```
cycle 214   latch on   running
```

Sista fältet är körläget — `running`, `paused` eller `live`.

Pause och live

`/pause` stoppar cykeln helt: ingen input, inga regler, ingen output. Prompten fungerar fortfarande, så du kan inspektera tillstånd och lägga till deklarationer eller regler; ombyggnaden skjuts upp till `/resume`.

`/live` fryser bara **reglerna**. Ingångar samplas fortfarande och utgångar skrivs fortfarande, så hårdvaran är kvar inkopplad medan programmet står still — vilket är vad man vill när man petar på pinnar för hand:

```
/live
> Led = 1          # actually lights the LED
> Btn              # reads the real pin
/resume
```

`/resume` lämnar båda lägena.

Redigera ett körande program

Deklarationer och regler kan läggas till när som helst — körande, pausad eller live. En ny regel kopplas in i den reaktiva grafen vid nästa cykelgräns, så den börjar fyra utan omstart. Det är det avsedda sättet att patcha ett levande system: eftersom den

sista regeln som skriver ett fält vinner, överrider en regel som läggs till vid prompten en tidigare.

Om en rad inte går att parsa behålls ingenting. Ett fel inne i en oavslutad `#module` rullar tillbaka **hela modulen** och rapporterar `Module aborted`, så raderna du skriver därefter inte tyst sväljs av en modul som aldrig kan stängas.

Slå av och på regler

`#disable` slår **av** en enskild regel via dess nummer; `#enable` slår på den igen. En avslagen regel hoppas över varje cykel — inget annat ändras.

`/list` numrerar reglerna i vänsterkolumnen och markerar en avslagen med `!`:

```
> /list
  1 R   Btn ? Led = 1
  2 R   Temp > 30 ? Fan = 1
> #disable 2
> /list
  1 R   Btn ? Led = 1
  2 R!  Temp > 30 ? Fan = 1     # av
> #enable 2                     # på igen
```

Du kan ange en lista eller ett intervall, och svepa med ett intervall:

```
#disable 3 5 7        # flera på en gång
#disable 2-6          # ett inklusivt intervall
#enable  1-99         # slå på allt igen (toppen klampas till sista regeln)
```

Vad den är bra för

- **Bygga sig förbi en bugg.** En regel bråkar — den slåss med en annan regel, snubblar på en trasig sensor, dränker en utgång. Istället för att stoppa hela programmet för att redigera och omflasha, slå av just den regeln och fortsätt köra:

  ```
  > #disable 4              # tysta den felande regeln
  > Fan = Temp > 25         # lägg till en korrigerad ersättningsregel
  ```

  Resten av programmet är orört. Du har *byggt dig förbi* buggen live och kan ta dig tid med en ordentlig fix.

- **Patcha firmware utan omflashning.** Regelnummer löper rakt genom de bakade ROM-reglerna och vidare in i dem du lägger till vid prompten, så `#disable 2` kan slå av en **firmware**-regel lika enkelt som en interaktiv. Tysta en ROM- regel, lägg en RAM-regel i dess ställe, och brädan beter sig på det nya sättet — ingen ombyggnad, ingen uppladdning.

- **Bisektera beteende.** Osäker på vilken regel som orsakar en effekt? Slå av ett intervall, bekräfta att det upphör, slå sedan på reglerna några i taget för att hitta boven.

- **Driftsättning och test.** Ta upp en maskin en regel i taget: slå av allt, slå på reglerna efter hand som varje delsystem verifieras.

Värt att veta

- En avslagning **följer sin regel**. Regelnummer förskjuts när du infogar eller tar bort en regel, men en avslagning kommer ihåg via identitet genom de ändringarna — den stannar på regeln du slog av, inte på vilket nummer den råkade ha.
- Avslagningar **består**. `/save` skriver dem till EEPROM och `/load` återställer dem, så en bräda kommer upp igen med samma regler avslagna. (Om programmet ändrats så mycket att numren inte längre stämmer släpps den sparade mängden med ett meddelande istället för att slå av fel regler.)
- `#disable 9` när det inte finns någon regel 9 är ett **fel** — att namnge en regel som inte finns är nästan alltid ett skrivfel. Ett *intervall* som svämmar över (`2-99`) läses som "till slutet" och klampas.
- De första 128 reglerna bär var sin brytare; regler bortom det körs alltid.

Direkt evaluering

I interaktivt läge kan du:

```
> X + 1              # evaluate expression
> X = 5              # assign value directly
#variable Y = 10     # add new declaration
Y = X * 2        # add new rule
```

Utgångslatch

Latchen styr om utgångar skrivs till hårdvara. Som en elektronisk latch:

- `/latch on` - Håll (latcha) nuvarande utgångsvärden. Utgångar beräknas internt men skrivs inte till pinnar.
- `/latch off` - Släpp latchen. Utgångar flödar igenom till hårdvara normalt.

Interaktivt läge startar med latch PÅ (utgångar frysta). Detta är en säkerhetsfunktion - du kan experimentera utan att påverka hårdvaran. Använd `/latch off` när du är redo att aktivera utgångar.

För programmatisk styrning i körande kod, använd `latch()`-funktionen:

```
latch(1) ? Fault       // hold outputs on fault condition
latch(0) ? !Fault      // release when fault clears
```

Arduino-exempel

Blink - Blinkande LED

Det klassiska Arduino-exemplet i CandySpeak:

```
#digital Led out 13
#timer Blink 500 = 1

Led = ~Led, Blink = 1 ? timeout(Blink)
```

LED växlar var 500:e millisekund. `Blink = 1` startar om timern.

Knapp med LED

Tänd LED när knappen trycks:

```
#digital Button in pullup 2
#digital Led out 13

Led = !Button
```

Knappen är aktiv-låg (pullup), så `!Button` är sant när den trycks.

Debouncer - Avfångare

Filtrera bort kontaktstuds från en knapp:

```
#module Debouncer
#timer T 50
#variable Raw in integer
#variable Out:1 out integer = 0
#variable Prev:1 integer = 0

T = 1 ? Raw != Prev
Prev = Raw, Out = Raw ? timeout(T)
#end

#digital Button in pullup 2
#digital Led out 13
#Debouncer db Raw <- !Button

Led = db.Out
```

Modulen väntar 50 ms efter en ändring innan den accepterar det nya värdet.

Toggle - Växla med knapp

Tryck för att växla LED:

```
#module Debouncer
#timer T 50
#variable Raw in integer
#variable Out:1 out integer = 0
#variable Prev:1 integer = 0

T = 1 ? Raw != Prev
Prev = Raw, Out = Raw ? timeout(T)
#end

#digital Button in pullup 2
```

```
#digital Led out 13
#variable LedState:1 = 0
#Debouncer db Raw <- !Button

LedState = ~LedState ? rising(db.Out)
Led = LedState
```

Dimmer - PWM-styrning

Justera ljusstyrka med två knappar:

```
#digital BtnUp in pullup 2
#digital BtnDown in pullup 3
#analog Dimmer:8 out pwm 9
#variable Brightness:8 = 128
#timer Rep 100

Brightness = Brightness + 10 ? !BtnUp && Brightness < 245
Brightness = Brightness - 10 ? !BtnDown && Brightness > 10
Rep = 1 ? !BtnUp || !BtnDown
Dimmer = Brightness
```

Temperaturstyrning

Enkel termostat med hysteres:

```
#analog Sensor in A0
#digital Heater out 7
#variable Setpoint = 200
#variable Hysteresis = 10

Heater = 1 ? Sensor < Setpoint - Hysteresis
Heater = 0 ? Sensor > Setpoint + Hysteresis
```

Värdet 200 motsvarar cirka 20C med en typisk NTC-sensor.

Rinnande ljus - Knight Rider

LEDs som rör sig fram och tillbaka:

```
#digital Led0 out 2
#digital Led1 out 3
#digital Led2 out 4
#digital Led3 out 5
#digital Led4 out 6
#digital Led5 out 7
#digital Led6 out 8
#digital Led7 out 9

#timer Step 100 = 1
```

```
#variable Pos:4 = 0
#variable Dir:1 = 0

Pos = Pos + 1, Dir = 1 ? timeout(Step) && !Dir && Pos < 7
Pos = Pos - 1, Dir = 0 ? timeout(Step) && Dir && Pos > 0
Dir = 1 ? Pos == 7
Dir = 0 ? Pos == 0
Step = 1 ? timeout(Step)

Led0 = (Pos == 0)
Led1 = (Pos == 1)
Led2 = (Pos == 2)
Led3 = (Pos == 3)
Led4 = (Pos == 4)
Led5 = (Pos == 5)
Led6 = (Pos == 6)
Led7 = (Pos == 7)
```

Pulserande LED - Andning

Mjuk pulsering med PWM:

```
#analog Led:8 out pwm 9
#timer Step 20 = 1
#variable Brightness:8 = 0
#variable Dir:1 = 0

Brightness = Brightness + 5 ? timeout(Step) && !Dir
Brightness = Brightness - 5 ? timeout(Step) && Dir
Dir = 1 ? Brightness >= 250
Dir = 0 ? Brightness <= 5
Step = 1 ? timeout(Step)
Led = Brightness
```

Trafikljus

Sekventiellt trafikljus:

```
#digital Red out 2
#digital Yellow out 3
#digital Green out 4

#timer Phase 1000 = 1
#variable State:2 = 0

State = 1 ? timeout(Phase) && State == 0
State = 2 ? timeout(Phase) && State == 1
State = 3 ? timeout(Phase) && State == 2
State = 0 ? timeout(Phase) && State == 3
```

```
Phase = 1 ? timeout(Phase)

Red = (State == 0 || State == 1)
Yellow = (State == 1 || State == 3)
Green = (State == 2)
```

Sekvens: Röd -> Röd+Gul -> Grön -> Gul -> Röd…

Samma maskin läser betydligt tydligare med namngivna `#states` — varje fas är ett tillstånd, och en övergång är en tilldelning till `State`:

```
#digital Red    out 2
#digital Yellow out 3
#digital Green  out 4
#timer   Phase  1000 = 1
#states  red redyellow green yellow

#in INIT
    State = red
#end
#in red
    Red=1, Yellow=0, Green=0
    State = redyellow ? timeout(Phase)
#end
#in redyellow
    Red=1, Yellow=1, Green=0
    State = green ? timeout(Phase)
#end
#in green
    Red=0, Yellow=0, Green=1
    State = yellow ? timeout(Phase)
#end
#in yellow
    Red=0, Yellow=1, Green=0
    State = red ? timeout(Phase)
#end

Phase = 1 ? timeout(Phase)      // restart the timer each phase
```

Varje tillstånd driver de tre lamporna och lämnar, vid `timeout(Phase)`, över till nästa tillstånd. Utgångsmönstret för en fas sätter sig cykeln efter att tillståndet trätt in — osynligt för ett ensekunders ljus, och strukturen skalar till betydligt fler tillstånd utan trasslet av `State == n`-villkor.

Baka in ett program i firmware

Ett CandySpeak-program kan *bakas in i firmware* i stället för att parsas vid uppstart. `-C` skriver det parsade programmet som C-källkod — `rom.c`, som definierar arrayerna `rom_decl[]`, `rom_instr[]` och `rom_str[]`:

```
./csp -C -n program.csp > rom.c
```

Denna `rom.c` **kompileras och länkas** in i firmware (den `#include`-as inte som en header). Vid uppstart pekar `csp_load_rom()` runtime på den, och programmet kör på plats ur flash.

Vad det ger — och inte ger. Det inbakade programmet kör på *samma* virtuella maskin över *samma* bytekod som ett program som skrivits in vid körning; det är **inte** snabbare (ocachade flash-läsningar kan till och med göra det en aning långsammare än ett RAM-resident program). De verkliga vinsterna ligger på annat håll:

- **Sparar RAM** — programmet ligger i flash, inte i det knappa RAM:et.
- **Ingen parsning vid boot** — programmet är klart direkt; parsern behöver inte ens länkas in.
- **Det är firmware** — det överlever utan att sparas till EEPROM.

Skriva en sketch

Det finns ingen autogenererad `.ino`-wrapper. Idag finns två praktiska vägar, båda med utgångspunkt i en fork/checkout av CandySpeak-källkoden:

1. **Hacka brädglue:t.** Redigera `csp_arduino.c` (plattformslagret: `csp_board_*`, `setup()`/`loop()`). Det är den enklaste vägen när man lägger till eller kopplar in hårdvara. Se `CandySpeak/CandySpeak.ino` för ett komplett, fungerande exempel (Circuit Playground Express) — det visar den verkliga cykeln: `csp_input()` → `csp_cycle()` → `csp_commit()` → `csp_output()`, inramad av `csp_rt_init` / `csp_load_rom` / `csp_rebuild` / `csp_setup(&state)` under `setup()`.
2. **Frys ett program.** Generera `rom.c` med `-C` som ovan, lägg in den i bygget och flasha. Sketchen laddar den via `csp_load_rom()`.

Firmware-bygget behöver kärnkällkoden — `csp.h`, `csp_config.h`, `csp_rt.c`, `csp_print.c`, `csp_strings.c`, `csp_parse.c`, `csp_eeprom.c`, `bitpack.h`, `csp_fixpoint.h` — plus ditt brädglue och, för den frysta vägen, `rom.c`.

> Detta är det nuvarande, handpåläggande arbetsflödet; en smidigare sketch-historia återstår att designa.

PDF-generering

Denna manual kan konverteras till PDF med pandoc:

```
# Install pandoc and LaTeX
sudo apt install pandoc texlive-xetex fonts-dejavu

# Generate PDF
pandoc doc/manual_sv.md -o doc/manual_sv.pdf \
  --pdf-engine=xelatex \
  --toc \
  --toc-depth=2 \
  -V colorlinks=true \
  -V linkcolor=blue
```

Appendix: Snabbreferens

Deklarationer

```
#variable <name>[:<bits>] [type] [= value]         // bits 1..32, typlös = tecknad
#variable  <name>:<bits>  [big|little]  bind  <buffer>[<a>..<b>]     //  bit-
field view
#digital <name> [in|out|inout] [pullup|pulldown] [<port>:]<pin>
#analog <name>[:<resolution>] [in|out] [pwm] [<port>:]<pin>
#timer <name> <period_ms> [= 1]
#constant <name> = <value>
#buffer <name>:<bytes> [type]           // shared storage (size in BYTES)
#buffer <name>:<bytes> [in|out] can <id>  // CAN frame (size in BYTES)
#field <name>:<bits> [type] [big|little] <frame>[<a>..<b>] // field of a frame
#states <name> ...                      // INIT/NORMAL/FAILSAFE implicit
#module <name> ... #end
```

Buffertar

```
#buffer Buf:3             // 3 BYTE delad lagring (storleken är alltid byte)
Buf[0] = 52                 // byte access (index = byte)
X = Buf[0..1]               // byte range -> one value (little-endian)

#variable F:8     bind Buf[0..7]    // bind range = bits
#variable G:10 big bind Buf[8..17]  // big-endian bit-field
```

CAN

```
#buffer F201:8 in  can 0x201     // 8 BYTES (the DLC), received
#buffer Out:8  out can 0x200     // transmitted
#field Speed:16 unsigned F201[0..15]   // field: a bit view into the frame

Speed = v                      // writing a field sends the frame (event PDO)
Out.tx  = 1 ? timeout(Beat)      // cyclic PDO: send even if unchanged
Out.dlc = 3                      // send 3 bytes instead of 8

println(Speed) ? F201.rx        // .rx: true the cycle the frame is readable
Got = F201.dlc                   // bytes the sender actually sent
Id  = F201.id                    // frame id

./csp --can=vcan0 prog.csp       // Linux: SocketCAN
```

Modulinstansiering

```
#<Module> <instance> [<init>]*

Init forms (can be mixed):
  field = value      // static init of the field's VALUE, runs once in INIT
```

```
  field.part = value // set a field attribute once
  field <- expr      // reactive connection (seeded once at start-up)

 D.pin = 2        // place a #digital/#analog  (NOT `D = 2`, that is a value)
  D.port = 1
  T.period = 500
```

Regler

```
<actions> ? <condition>
action1, action2 ? condition
variable = expression ? condition   // regular assignment
variable <- expression              // reactive (runs on change)
variable <- expression ? condition  // reactive with condition
Timer = 1 ? condition               // start timer
```

Timer-funktioner

```
timeout(T)    // true one cycle at timeout
elapsed(T)    // ms since start
progress(T)   // 0-100% of period
T = 1         // start/restart timer
```

Kantdetektering

```
changed(x)    // value changed
rising(x)     // 0 -> 1
falling(x)    // 1 -> 0
```

States

```
#states A B C                // INIT, NORMAL och FAILSAFE finns alltid

#in A                        // rules active only while State == A
    ...
    State = B ? cond         // transition
#end

#in FAILSAFE                 // den säkra ön: dit en gång, aldrig därifrån
    ...                      // (bara en reset släpper den)
#end
```

Parts

```
<resource>.<part>            // read or write an attribute
D.pin = 17                   // .val .pin .port .dir .pwm
T.period = 500               // .pullup .pulldown .period .fired
Ready = 1 ? T.fired
```

```
F.id  F.rx  F.tx  F.dlc      // CAN frame parts (also via any of its fields)
```

Interaktivt

```
/pause  /resume  /live        // freeze all / continue / freeze rules only
/latch on|off                 // hold or release outputs
/list  /state  /memory        // inspect
/save  /load                  // EEPROM
#disable 2   #enable 2        // slå av / på en regel via nummer
#disable 2-6                  // en lista eller ett inklusivt intervall
```

Packa och packa upp

```
Buf <<= A B C             // pack fields into a buffer (ascending bits)
Buf <<= (X+4):3 X:2 6:3   // field := <expr>[:<bits>]
Buf >>= RA:3 RB:2 RC:3    // unpack: each var takes the next field
```